

MATH FOR NEUROSCIENCE & MACHINE LEARNING

Textbook Edition

Vectors, Neural Population States, Feature Spaces, and Dynamic Trajectories

Lesson 1

A step-by-step textbook companion to the Python notebook.

Created by

Peter Adepoju

`petera@aims.ac.za`

The Motivation for This Course

The motivation for creating this course came from a long-standing gap observed. Many students without a strong quantitative background, especially those in the life sciences, struggle with mathematical content. When faced with an equation-heavy material, some feel lost; others decide that the mathematics is not relevant and skip it entirely.

Surprisingly, students from physics, engineering, and even mathematics can experience the same frustration. The root cause is rarely lack of intelligence or curiosity. More often, the mathematics is introduced in a fully abstract or theoretical form, without practical demonstrations or real-world context that would make the ideas immediately meaningful to these students.

Yet mathematics and statistics are the bedrock of computational neuroscience and machine learning. Nearly every lecture or tutorial in these fields contains substantial mathematical content, from differential equations describing dynamical systems, to linear algebra for dimensionality reduction, probability theory for Bayesian inference, and optimisation algorithms for learning. Every neuron model, every learning algorithm, and every analysis of neural data is fundamentally built on mathematical structures. The aim of this program is therefore to bring that mathematics to life.

We will achieve this by computing every concept in real time using the Python programming language. Every formula encountered in this course will be translated into runnable code; abstract equations will become concrete numbers, plots, and simulations that reveal neurons firing, learning curves evolving, and population dynamics unfolding. You will understand clearly why each mathematical idea matters. By the end of this program, mathematics will no longer appear abstract; it will be a tool you can actively use to think, compute, and model.

Contents

The Motivation for This Course	i
1 Vectors	1
1.1 Motivation	1
1.2 Brain State as a Vector	1
1.3 Scalars and Different Kinds of Vectors	2
1.4 Vectors in NumPy	2
2 Neural Population Geometry	4
2.1 Constructing Repeated Trials Under Several Conditions	4
2.2 2D State Space	4
2.3 3D State Space	5
3 Vector Operations	6
3.1 Vector Addition, Subtraction, and Scaling	6
3.2 Norm, Magnitude, and Euclidean Distance	7
3.3 Unit Vectors and Normalisation	8
3.4 Why the Zero Vector Is Rejected	8
3.5 Linear Combinations and Coordinates	9
3.6 Span, Independence, and Dependence	10
4 Time Series for Neural Trajectories	11
4.1 Building a Synthetic Time-Varying Recording	11
4.2 Firing-Rate Traces as Time Series	12
4.3 The Corresponding 3D Population Trajectory	12
Reference Material	13

1 Vectors

1.1 Motivation

Computational neuroscience studies nervous systems using mathematical models, data, and computation. Machine learning studies algorithms that learn useful patterns or mappings from data. These fields ask different questions, but they often begin with the same data structure: an ordered collection of numbers.

For example, imagine that three neurons are measured at the same time. Their firing rates may be 12, 4, and 19 spikes per second. We could keep these as three separate variables, but that would hide the fact that they jointly describe one population state. Instead, we write

$$\mathbf{r} = (12, 4, 19) \text{ spikes/s.}$$

This is a **vector**.

A machine-learning observation has the same mathematical structure. A house might be represented by size, number of bedrooms, and age:

$$\mathbf{x} = (1416, 3, 40).$$

where coordinate 1 is size in square feet, coordinate 2 is bedrooms, and coordinate 3 is age in years. The numbers have different units.

Simple Intuition

A vector is like a labelled container. The container holds several numbers, but the order and meaning of the slots are fixed. Slot 1 must mean the same thing every time the vector is used.

1.2 Brain State as a Vector

Suppose three neurons are recorded simultaneously under two conditions:

$$\mathbf{r}_1 = (12, 4, 19), \quad \mathbf{r}_2 = (50, 8, 33).$$

Then

$$\Delta \mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1 = (38, 4, 14)$$

and

$$\|\Delta \mathbf{r}\|_2 = \sqrt{38^2 + 4^2 + 14^2} \approx 40.69 \text{ spikes/s.}$$

The subtraction tells us how each neuron changed. The norm compresses the three-coordinate change into a nonnegative number representing Euclidean movement through the three-neuron state space.

Neuro / ML Context

A population vector is a snapshot. It answers: “What are all recorded neurons doing at this moment?”
A trajectory, introduced later, is a sequence of snapshots ordered through time.

1.3 Scalars and Different Kinds of Vectors

Definition: Scalar

A scalar is one number. Examples include one neuron's firing rate, one temperature, one loss value, or one learning rate.

Definition: Vector

A vector is an ordered collection of numbers treated as one object. An n -coordinate vector belongs to $\mathbb{R}^n$ when every coordinate is a real number.

1.3.1 Three Views of One Vector

The vector $\mathbf{v} = (4, -2)$ can be understood in three ways.

1. **An ordered list:** As a list, the vector is simply the two coordinates 4 and -2 . This view is most useful for storage, indexing, and computation.
2. **A displacement arrow:** As an arrow beginning at the origin, it means move 4 units along the first axis and 2 units in the negative direction of the second axis. This view emphasises change.
3. **A point:** As a point, it identifies the location $(4, -2)$. This view emphasises state rather than motion.

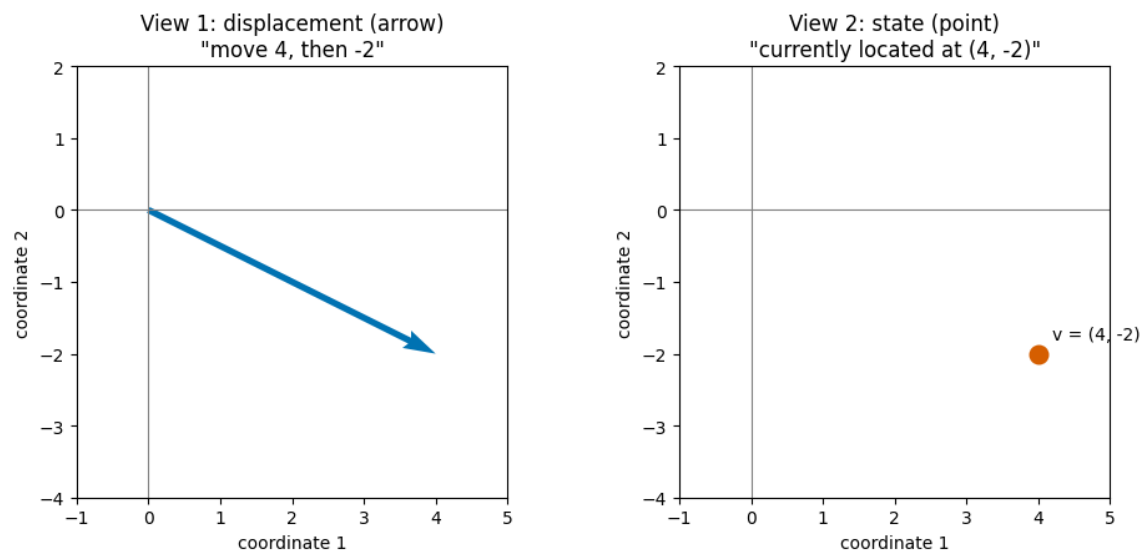


Figure 1: The same coordinates interpreted as a displacement arrow and as a state-space point.

1.4 Vectors in NumPy

```
1 neuron_labels = ['N1', 'N2', 'N3', 'N4', 'N5']
2 r = np.asarray([12.5, 45.3, 8.7, 33.1, 71.4])
```

This array has shape $(5,)$, one dimension, five entries, and floating-point type. `np.asarray` converts its input to an array when needed.

1.4.1 Using *zip* to join labels and values

```
1 for label, rate in zip(neuron_labels, r):
2     print(f'{label}: {rate:.1f} spikes/s')
```

`zip` pairs the first label with the first value, the second label with the second value, and so on.

1.4.2 Shape Conventions

Three arrays can contain the same visible numbers but have different shapes:

```
1 v_flat = np.array([1.0, 2.0, 3.0])
2 v_col = v_flat.reshape(3, 1)
3 v_row = v_flat.reshape(1, 3)
```

`v_flat.shape = (3,)`, `v_col.shape = (3,1)`, `v_row.shape = (1,3)`.

The flat array has one axis. The column and row objects each have two axes and are therefore matrices, even though one dimension has length 1.

1.4.3 Broadcasting

Adding `v_flat` and `v_col` produces a 3×3 array:

Expected Output and Meaning

```
[[2.  3.  4.]
 [3.  4.  5.]
 [4.  5.  6.]]
```

NumPy aligns shapes from the right. The flat shape `(3,)` behaves like `(1,3)` for broadcasting, while the column has shape `(3,1)`. One dimension expands down the rows and the other expands across columns. The result is an outer grid of pairwise sums.

Before combining arrays, ask:

1. Do they have the same number of coordinates?
2. Do those coordinates represent the same variables in the same order?
3. Are their shapes exactly what I intend?

2 Neural Population Geometry

2.1 Constructing Repeated Trials Under Several Conditions

From the notebook, we have four mean response vectors:

```
1 condition_means = {
2     'Baseline': np.array([5.0, 8.0, 6.0]),
3     'Stimulus A': np.array([55.0, 12.0, 9.0]),
4     'Stimulus B': np.array([10.0, 60.0, 45.0]),
5     'Motor': np.array([28.0, 32.0, 60.0]),
6 }
```

Fifteen noisy trials are generated around each mean:

```
1 condition_trials = {
2     cond: mean + RNG.normal(0.0, trial_noise_sd, size=(n_trials, 3))
3     for cond, mean in condition_means.items()
4 }
```

The normal distribution has mean 0 and standard deviation 4 spikes/s. The resulting noise array has shape (15,3). Adding a length-3 mean vector broadcasts the mean across the fifteen rows.

2.2 2D State Space

A state space uses one axis per coordinate. With three neurons, the full ambient state space is three-dimensional. The first plot chooses only N1 and N2 so it can be displayed in an ordinary plane.

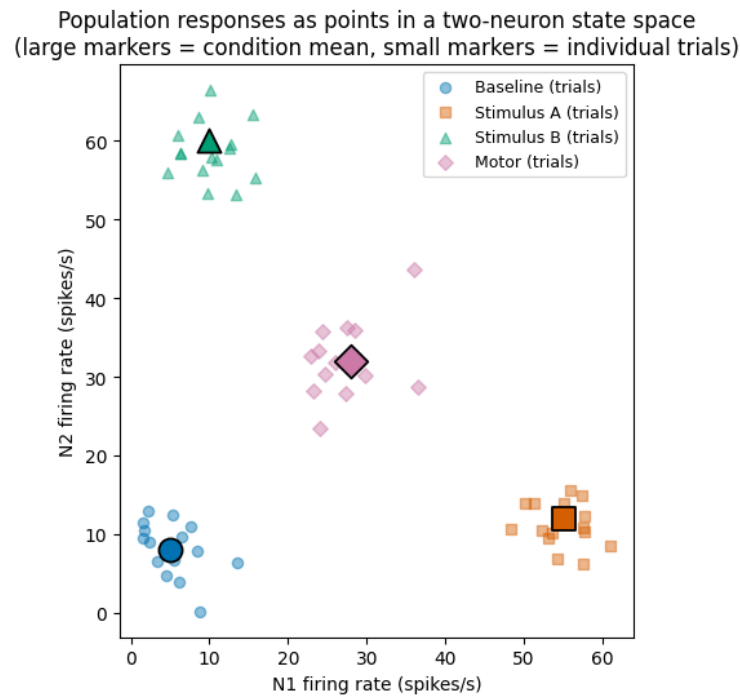


Figure 2: Small markers are individual noisy trials; large markers are the condition means.

Each trial is a point (r_1, r_2) . Points from the same condition form a cloud because the simulated measurements vary from trial to trial. The large marker shows the theoretical mean used to generate that cloud.

2.3 3D State Space

The 3D plot includes all three neurons.

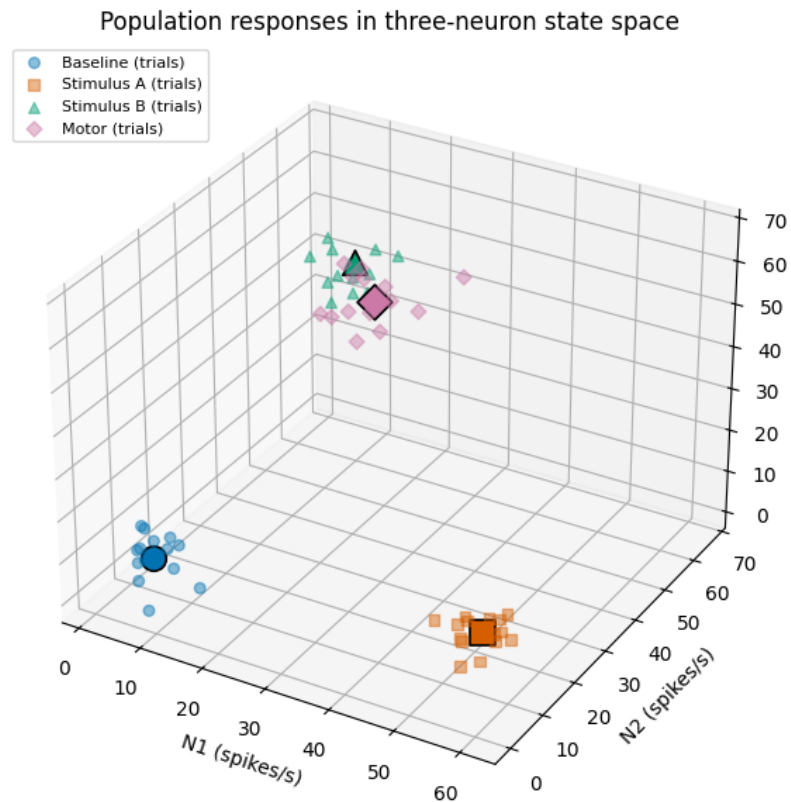


Figure 3: All three coordinates are shown. The perspective can make some separations appear larger or smaller, which is why an interactive rotatable version is useful.

2.3.1 Ambient dimension and display dimension

Definition: Ambient dimension

The ambient dimension is the true number of coordinates in the representation. A recording of 100 neurons has ambient dimension 100.

Definition: Display dimension

The display dimension is the number of coordinates shown in one figure. A two-dimensional plot of a 100-neuron recording has display dimension 2, not ambient dimension 2.

Displaying fewer coordinates discards information unless a principled dimensionality-reduction method is used. We will cover such methods later.

3 Vector Operations

3.1 Vector Addition, Subtraction, and Scaling

For vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and scalar α ,

$$(\mathbf{u} + \mathbf{v})_i = u_i + v_i, \quad (\mathbf{u} - \mathbf{v})_i = u_i - v_i, \quad (\alpha \mathbf{v})_i = \alpha v_i.$$

Using Baseline $\mathbf{r}_A = (5, 8, 6)$ and Stimulus A $\mathbf{r}_B = (55, 12, 9)$:

$$\mathbf{r}_A + \mathbf{r}_B = (60, 20, 15), \quad \mathbf{r}_B - \mathbf{r}_A = (50, 4, 3), \quad 1.5\mathbf{r}_A = (7.5, 12, 9).$$

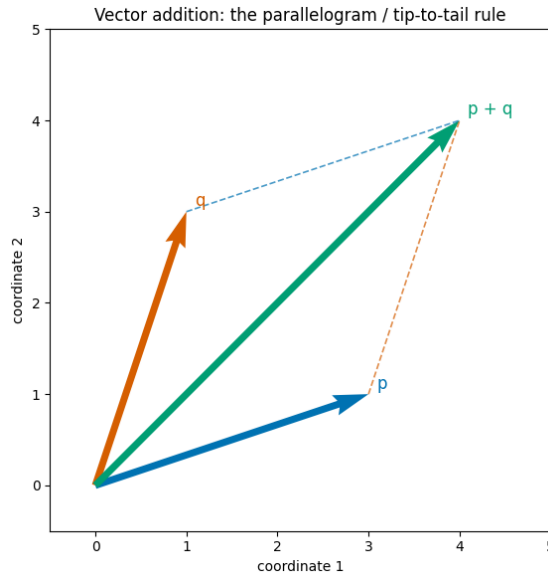


Figure 4: The parallelogram rule. The sum is obtained by constructing the diagonal of a parallelogram.

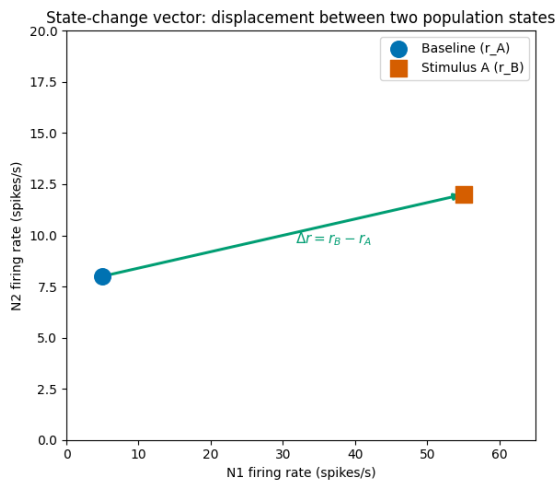


Figure 5: The displacement from Baseline to Stimulus A in the N1–N2 display plane.

The full change has three coordinates, but the plot displays only the first two. N1 increases by 50 spikes/s, N2 by 4, and N3 by 3. The visual arrow is therefore dominated by horizontal movement.

3.2 Norm, Magnitude, and Euclidean Distance

The Euclidean norm of $\mathbf{v} = (v_1, \dots, v_n)$ is

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2}.$$

For (3,4), the norm is 5. This is the Pythagorean theorem.

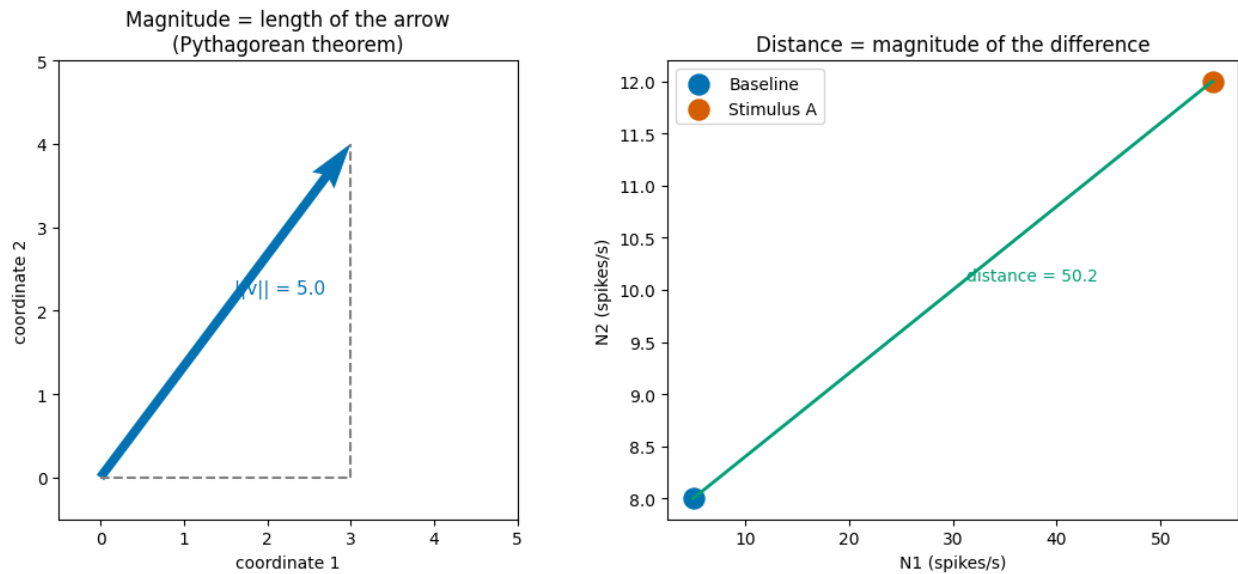


Figure 6: **Left:** magnitude is the length from the origin to a vector. **Right:** distance is the length of the difference between two points.

The distance between $\mathbf{u}$ and $\mathbf{v}$ is

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2.$$

Distance is the norm of a displacement.

3.2.1 The pairwise distance table

The diagonal is zero because each vector has zero distance from itself. The table is symmetric because $d(\mathbf{u}, \mathbf{v}) = d(\mathbf{v}, \mathbf{u})$.

	Baseline	Stimulus A	Stimulus B	Motor
Baseline	0.0	50.2	65.2	63.4
Stimulus A	50.2	0.0	75.0	61.1
Stimulus B	65.2	75.0	0.0	36.5
Motor	63.4	61.1	36.5	0.0

3.2.2 Why raw Euclidean distance can mislead

Euclidean distance treats one numerical unit on every axis as equally important. If N_1 varies between 0 and 200 while N_2 varies between 0 and 5, N_1 can dominate. The same problem occurs with house size in square feet and bedroom count. Scaling, standardisation, covariance-aware distances, and domain-specific metrics may be more suitable, depending on the question.

3.3 Unit Vectors and Normalisation

A unit vector has length 1. For nonzero $\mathbf{v}$,

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}.$$

For $(3,4)$, $\hat{\mathbf{v}} = (0.6, 0.8)$.

```

1 def normalise(v, eps=1e-12):
2     v = np.asarray(v, dtype=float)
3     norm = np.linalg.norm(v)
4     if norm < eps:
5         raise ValueError('The zero vector has no defined direction.')
6     return v / norm

```

3.4 Why the Zero Vector Is Rejected

The zero vector has magnitude zero. Dividing by zero is undefined, and there is no unique direction to preserve. The tolerance `eps` also protects against extremely tiny values that are numerically indistinguishable from zero.

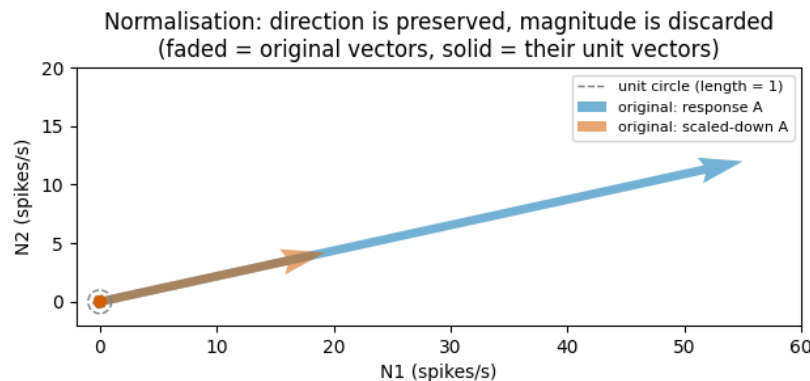


Figure 7: Two vectors with the same direction but different lengths become the same unit vector.

Normalisation removes overall response strength and keeps the relative direction. Whether that is desirable depends on the scientific question. If absolute firing strength matters, normalisation discards important information.

3.5 Linear Combinations and Coordinates

A linear combination is a weighted sum:

$$\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \cdots + \alpha_k \mathbf{v}_k.$$

The standard basis in $\mathbb{R}^2$ is

$$\mathbf{e}_1 = (1, 0), \quad \mathbf{e}_2 = (0, 1).$$

Any vector (v_1, v_2) can be written

$$\mathbf{v} = v_1 \mathbf{e}_1 + v_2 \mathbf{e}_2.$$

Thus, coordinates are the weights needed to reconstruct the vector from the basis.

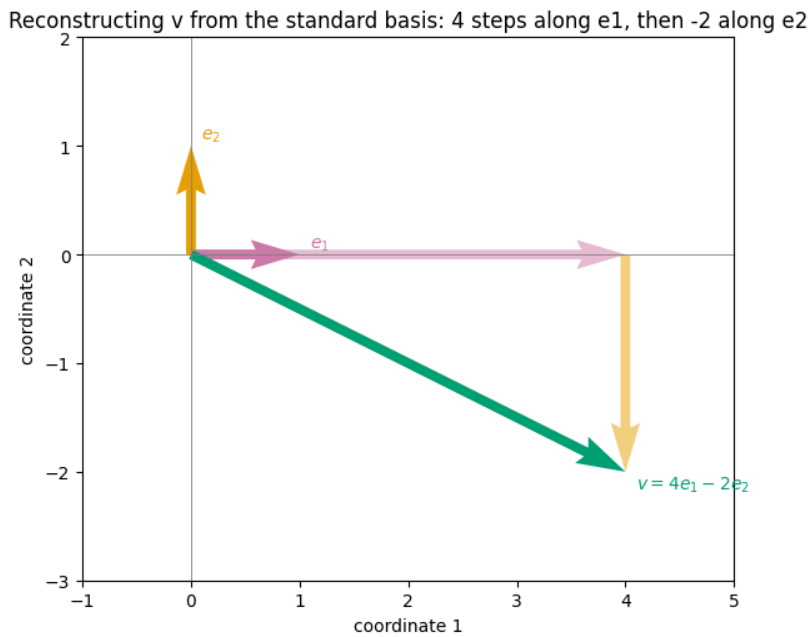


Figure 8: Reconstructing $(4, -2)$ as $4\mathbf{e}_1 - 2\mathbf{e}_2$.

3.5.1 Neural patterns as building directions

The notebook defines

$$\mathbf{u}_{\text{stim}} = (1, 1, 0), \quad \mathbf{u}_{\text{motor}} = (0, 1, 1).$$

With weights $a = 20$ and $b = 15$,

$$20\mathbf{u}_{\text{stim}} + 15\mathbf{u}_{\text{motor}} = (20, 35, 15).$$

This representation uses two coefficients instead of three raw rates. It is useful only if the two patterns genuinely capture the relevant activity.

3.6 Span, Independence, and Dependence

The span of a set of vectors is every point reachable by choosing any scalar weights and forming linear combinations. Two nonparallel vectors in 2D are linearly independent and span the whole plane. If one is a multiple of the other, they are dependent and span only one line.

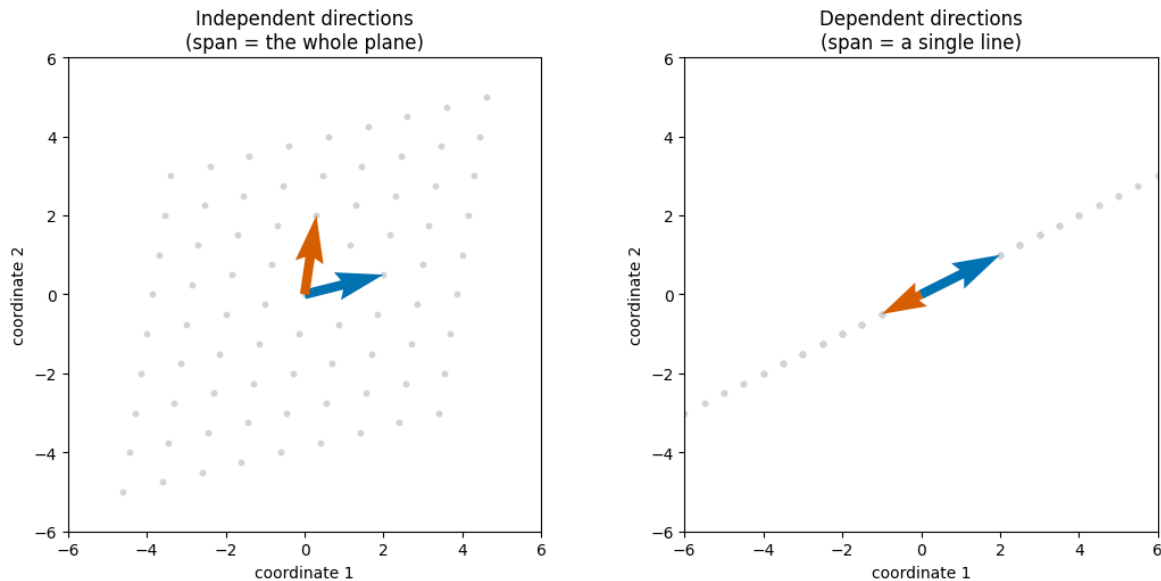


Figure 9: Independent directions create a plane of possibilities; dependent directions collapse to a line.

Simple Intuition

Independent vectors contribute genuinely new directions. A dependent vector is redundant: it can already be built from the others.

This idea leads toward dimensionality reduction. A recording may contain hundreds of neurons but vary mainly along a small number of coordinated directions. Estimating those directions is the main goal of methods such as PCA and latent dynamical models.

4 Time Series for Neural Trajectories

4.1 Building a Synthetic Time-Varying Recording

The notebook simulates three seconds of activity using 60 time bins of width 0.05 seconds:

```
1 dt = 0.05
2 n_bins = 60
3 time_axis = np.arange(n_bins) * dt
4 stim_onset_bin = 20
```

The final time is $59 \times 0.05 = 2.95$ seconds because indexing begins at zero. The stimulus begins at bin 20, corresponding to 1.00 second.

4.1.1 The sigmoid ramp

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

The sigmoid transitions smoothly from values near 0 to values near 1. The notebook shifts and scales the input so that the transition occurs around stimulus onset:

```
1 ramp = sigmoid((np.arange(n_bins) - stim_onset_bin) / 3.0)
```

Dividing by 3 spreads the transition over several bins. A smaller divisor would make the rise steeper; a larger divisor would make it slower.

4.1.2 The three neuron traces

$$N_1(t) = 5 + 45 \text{ramp}(t), \quad N_2(t) = 10 + 8 \sin\left(\frac{2\pi t}{1.5}\right), \quad N_3(t) = 8 + 40 \text{ramp}(t) (1 - 0.5 \text{ramp}(t)).$$

N_1 rises after the stimulus. N_2 oscillates throughout the trial. N_3 rises and then partially settles because the quadratic expression changes as the ramp approaches 1.

4.1.3 Adding noise, stacking, and clipping

```
1 noise = RNG.normal(0.0, 1.5, size=(n_bins, 3))
2 population_recording = np.stack(
3     [n1_trace, n2_trace, n3_trace], axis=1
4 ) + noise
5 population_recording = np.clip(population_recording, 0, None)
```

`np.stack(..., axis=1)` places the three length-60 traces into three columns, producing shape $(60, 3)$. Each row is one time point; each column is one neuron. Noise makes the example look more like a recording. Clipping prevents negative firing rates.

4.2 Firing-Rate Traces as Time Series

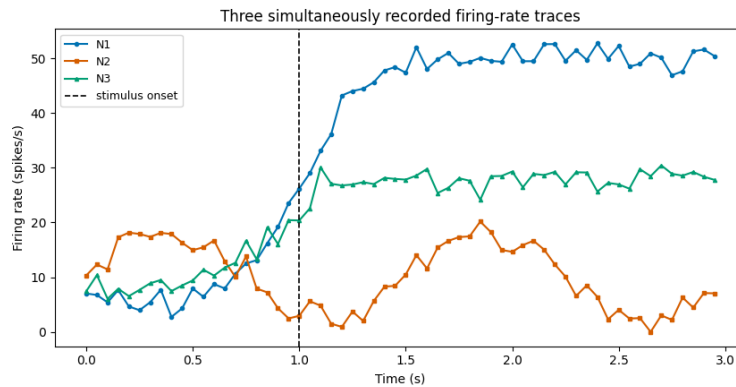


Figure 10: Each curve shows one neuron's firing rate as a function of time.

This figure answers “How does each neuron change over time?”

4.3 The Corresponding 3D Population Trajectory

At every time bin t_k , the three rates form one vector

$$\mathbf{r}(t_k) = (r_1(t_k), r_2(t_k), r_3(t_k)).$$

Plotting these vectors in chronological order produces a trajectory.

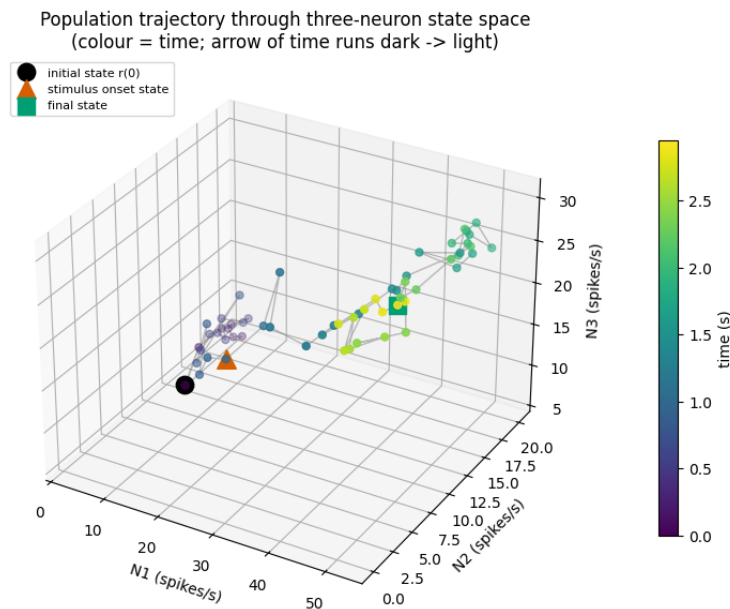


Figure 11: A trajectory through a three-neuron state space. Colour encodes time from dark to light.

The black circle is the initial state, the triangle marks stimulus onset, and the square marks the final state. The grey line connects consecutive time bins.

Reference Material

Glossary

Term	Definition
Ambient dimension	The number of coordinates in the full representation.
Animation frame	One visual state in a sequence of changing plots.
Basis	A nonredundant set of directions that can build every vector in a space.
Broadcasting	NumPy's rule for expanding compatible shapes during element-wise operations.
Coordinate	One ordered component of a vector.
Display dimension	The number of dimensions shown in a figure.
Feature vector	All measured features for one observation.
Linear combination	A weighted sum of vectors.
Linear dependence	A vector can be constructed from the others.
Norm	A rule that measures vector magnitude.
Normalisation	Scaling a nonzero vector to length 1.
Population state	Simultaneous activity values across recorded neurons.
Scalar	One number.
Span	All vectors reachable by linear combinations of a set.
State space	A coordinate space whose points represent system states.
Trajectory	A time-ordered path through state space.
Vector	An ordered collection of numbers treated as one object.

Further Reading

1. Deisenroth, M. P., Faisal, A. A., and Ong, C. S. *Mathematics for Machine Learning*. Cambridge University Press. <https://mml-book.com>
2. Neuromatch Academy. *Computational Neuroscience Course*. <https://compneuro.neuromatch.io/>
3. Strang, G. MIT OpenCourseWare 18.06 *Linear Algebra*. <https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/>
4. NumPy documentation. <https://numpy.org/doc/stable/>
5. Matplotlib documentation. <https://matplotlib.org/stable/>
6. Plotly Python documentation. <https://plotly.com/python/>